



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE24CS151B**

***Lecture #2
Error handling in File Handling***

**By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: File Handling and Portable Programming****Topic: Error handling in File Handling**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

Errors are the problems or the faults that occur in the program, which makes the behavior of the program abnormal, and experienced programmers and developers can also make these faults. **Programming errors** are also known as the **bugs or faults**.

When working with files in C, it's crucial to ensure that operations such as **opening, reading, writing, or closing a file are executed safely**. However, various factors such as **missing files, access restrictions, or disk issues can cause these operations to fail**. If such failures are not handled properly, they may lead to incorrect program behavior, crash, or data loss. The most common errors that may occur are:

1. **Reading beyond the end-of- file:** EOF is a condition in a computer operating system where no more data can be read from a data source. The data source is usually a file or stream.
2. **Performing operations on the file that has not still been opened:** Before performing any operation on a file, you must first open it. An attempt to perform operations on a file that has not been opened yet will lead to error
3. **Writing to a file that is opened in the read mode:** A mode is used to specify whether you want to open a file to perform operations like read, write append etc.
4. **Opening a file with invalid filename:** If you attempt to upload a file that has an invalid file name or type you will receive error.
5. **Write to a write-protected file:** Write-protected file is the ability to prevent new information from being written or old information being changed. In other words, information can be read, but nothing can be added or modified.

Error handling allows the program to **detect if a file operation failed, respond gracefully (e.g., display an error message, attempt recovery, or exit safely)** and **maintain robustness and user-friendliness**. It helps during the processing of input-output statements by catching severe errors that might not otherwise be noticed. In C, standard library functions provide return values and flags that can be checked to identify and manage such issues effectively.

Types of Error Handling in C

C does not have built-in exception handling features. Instead, **it uses manual error handling techniques**. These can be broadly categorized into:

1. **Return value checking (Most Common in C):** Most functions return a value that indicates whether they succeeded or failed. These values must be explicitly checked to detect errors and handle them properly. This helps avoid unexpected behavior or program crashes by ensuring that operations only proceed when they are safe. This practice also simplifies the debugging process by making it easier to identify where and why something went wrong.

Case 1: `fopen()` returns a NULL pointer on failure.

```
FILE *fp = fopen("data.txt", "r");
if (fp == NULL) {
    printf("Error opening file");
    exit(1);
}
```

Case 2: `fscanf()` returns the number of items successfully read. Check if the expected number of items was read.

```
int num;
if (fscanf(fp, "%d", &num) != 1) {
    fprintf(stderr, "Error reading integer from file\n");
}
```

Case 3: closing File safety

```
if (fclose(fp) != 0) {
    perror("Error closing file");
}
```

PS: Think of return code checking like **checking a receipt after shopping** — it helps you know if everything went right or if there was a billing error.

2. **Global Variable errno:** C programming being a system programming language, it provides you access at lower level in the form of return values from the functions. Most of the C function calls return -1 or NULL in case of any error and set an error code **global variable errno**. When a function is called in C, a variable named as `errno` is automatically assigned a code (value) which can be used to identify the type of error that has been encountered. Its a global variable **indicating the error occurred during any function call** and defined in the header file **errno.h**. Different codes (values) for `errno` mean different types of errors.

errno value	Type of Error
0	No Error
1	Operation not permitted
2	No such file or directory
5	I/O error
7	Argument list too long
9	Bad file number
11	Try again
12	Out of memory
13	Permission denied

The `errno` value indicate the types of error encountered. If you want to show the **error description**, there are **two functions that can be used to display a text message that is associated with `errno` - `perror()` and `strerror()`**

3. **`perror()` function:** The function `perror()` **stands for print error**. In case of an error, the programmer can determine the type of error that has occurred using the `perror()` function. It displays **the string you pass to it, followed by a colon, a space, and then the textual representation of the current `errno` value**.

Syntax: `void perror (const char *str) // (str: is a string containing a custom message to be printed before the error message itself.)`

Coding Example_1: Demo of perror()

```
#include <stdio.h>
#include <errno.h>
#include <string.h>
int main (){
    FILE *fp = fopen ("data.txt", "r"); // File with this name exist
    //FILE *fp = fopen ("data1.txt", "r"); // File with this name doesn't exist
    printf("Value of errno: %d\n", errno);
    perror("Bad code");
    return 0;
}
```

```
C:\Users\Dell>a
Value of errno: 0
Bad code: No error
```

```
C:\Users\Dell>a
Value of errno: 2
Bad code: No such file or directory
```

4. **strerror() function:** The function used to get a **human-readable error message**. Returns a pointer to the textual representation of the current error code usually stored in the global variable `errno`. It is important to note that the **returned string is statically allocated** and should not be modified.

Syntax: `char *strerror(int errnum);`

Coding Example_2: Demo of strerror()

```
#include <stdio.h>
#include <string.h>
#include <errno.h>
int main() {
    FILE *fp = fopen("data1.txt", "r"); // Trying to open a file that may not exist
    if (fp == NULL) {
        printf("File open failed: %s\n", strerror(errno));
    }
    return 0;
}
```

```
C:\Users\Dell>a
File open failed: No such file or directory
```

Coding Example_3: Usage of perror() and strerror() while handling illegal operations on the file

```
#include <stdio.h>
#include <errno.h>
#include <string.h>
int main ()
{
    FILE *fp;
    fp = fopen ("data.txt", "r"); // File with this name exist
```

```
C:\Users\Dell>a
value of errno is 9 with info:Bad file descriptor
Bad code: Bad file descriptor
```

```
fputc('A',fp); // writing to a file which is opened for read
printf("value of errno is %d with info:%s\n",errno,strerror(errno));
perror("Bad code");
fclose(fp);
return 0;
}
```

To ensure safe and error-free file operations in C, two standard library functions are provided to monitor the status of a file stream. These functions help detect specific conditions such as reaching the end of a file or encountering an error during input/output operations. The two commonly used status-checking functions are:

1. **fEOF()** – This function checks if the end of the file (EOF) has been reached. It is typically used in loops to prevent reading past the file's end.
2. **ferror()** – This function checks whether an error has occurred during a file operation, such as reading from or writing to the file. It helps identify issues like invalid file access or hardware failures.

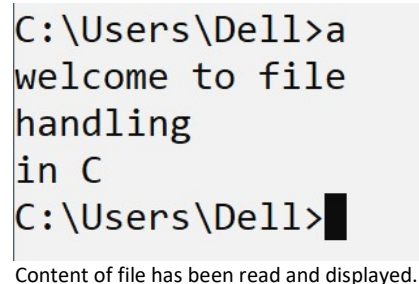
fEOF():

In C, `getc()` returns EOF when end of file is reached. `getc()` also returns EOF when it fails. So, only comparing the value returned by `getc()` with EOF is not sufficient to check for actual end of file. To solve this problem, C provides `fEOF()` function used to check whether the end of a file has been reached during a file read operation. This **returns a non-zero value only if end of file has reached, otherwise it returns 0.**

Syntax: `int fEOF(FILE *stream);`

Coding Example_4: Usage of fEOF() in loops to detect when a read operation failed.

```
#include <stdio.h>
int main ()
{
    FILE *fp = fopen("data.txt", "r");
    char ch;
    if (fp != NULL) {
        while (!fEOF(fp)) {
            ch = fgetc(fp);
            if (!fEOF(fp)) // check again after reading
                putchar(ch);
        }
    }
}
```



```
        }  
    }  
    fclose(fp);  
    return 0;  
}
```

feof() does not predict or detect EOF in advance. Instead, it checks if the last file operation **attempted to read beyond the end of the file. Best Practice is** instead of checking feof() before reading, check the return value of the read function itself (like fgetc() or fscanf()), and use feof() only to confirm EOF after a failed read.

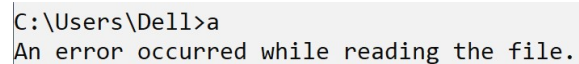
ferror():

This function checks for any error in the stream occurred during input or output operations. **It returns a value zero if no error has occurred and a non-zero value if there is an error.** The error indication will last until the file is closed unless it is cleared by the clearerr() function.

Syntax: int ferror (FILE *file_pointer);

Coding Example_5: Usage of ferror() to find the illegal operation on the file stream.

```
#include <stdio.h>  
int main() {  
    FILE *fp = fopen("data1.txt", "w"); // file is opened in write mode  
    char buffer[100];  
    if (fp != NULL) { // trying to read - illegal operation on the stream  
        if (fgetc(buffer, sizeof(buffer), fp) != NULL) {  
            printf("Read line: %s", buffer);  
        } else {  
            if (ferror(fp)) {  
                printf("An error occurred while reading the file.\n");  
            }  
        }  
    }  
    fclose(fp);  
}  
return 0;  
}
```



The `ferror()` is useful when a read or write operation fails. Instead of assuming the cause is end-of-file (EOF), this function helps determine whether the failure was due to an actual error such as:

- Reading from a corrupted file
- Writing to a full disk
- Accessing a file in an invalid mode

Best practice is that **the `ferror()` should only be used after performing input or output operations** such as `fgets()`, `fscanf()`, `fread()`, or `fwrite()`. It checks whether an error occurred during those operations and returns a non-zero value if an error has indeed happened. **It does not initiate the check by itself—it must follow a failed I/O attempt.** Additionally, because `ferror()` only signals that an error occurred without giving details, it's **recommended to use it in combination with `perror()` or `strerror(errno)` to retrieve a human-readable error message.** This is particularly helpful for debugging or logging errors for user-friendly messages. To distinguish between **an end-of-file condition and a real error**, it's also good practice to use `feof()` alongside `ferror()`.

Coding Example_6: Displaying the contents of the file on the terminal.

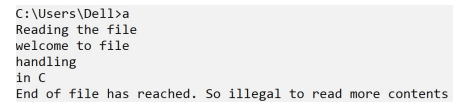
Version 1: Reading after encountering EOF

```
#include<stdio.h>
int main( )
{
    FILE *fp; char c;
    fp = fopen ( "data.txt", "r" ); // opening an existing file
    if ( fp != NULL )
    {
        printf( "Reading the file\n" );
        while ( !feof(fp) ) // returns nonzero if EOF encountered
        {
            c = fgetc(fp) ; // reading the file
            printf ("%c",c) ;
        }
        c = getc(fp); // doesn't show any error. so better to use feof function. See Version 2
        perror("\nBAD Code");
        fclose ( fp ) ; // Closing the file
    }
    return 0;
}
```

```
C:\Users\Dell>a
Reading the file
welcome to file
handling
in C
BAD Code: No error
```

Version 2:

```
printf( "Reading the file\n" );
while ( !feof(fp) ) // returns nonzero if EOF encountered
{
    c = fgetc(fp) ; // reading the file
    printf ("%c",c) ;
}
if(feof(fp))
{
    c = getc(fp);
    printf("\nEnd of file has reached. So illegal to read more contents\n");
}
fclose ( fp ) ; // Closing the file
```



```
C:\Users\Dell>a
Reading the file
welcome to file
handling
in C
End of file has reached. So illegal to read more contents
```

clearerr():

It is used to reset **the error and EOF indicators for a file stream**. After an error or end-of-file condition occurs on a stream, further operations might not proceed as expected until these indicators are cleared.

Coding Example_7: Writing to a file which is opened in read mode

```
#include <stdio.h>
#include <errno.h>
int main(){
    FILE* fp;
    char feedback[100];
    fp = fopen("data.txt", "r"); // opened in r mode. File must be existing
    printf("Provide feedback on the article: ");
    fgets(feedback, 100, stdin);
    for (int i = 0; i < feedback[i]; i++)
        fputc(feedback[i], fp); // writing to a file character by character
    printf("%d",errno);
    perror("Bad code"); //writing to a file is not allowed as the mode used is r while opening
    if (ferror(fp))
        printf("\nError writing to a file");
```

clearerr(fp); // error indication is stopped . next ferror() is returning 0

```
printf("\nAfter clearing fp\n");  
if (ferror(fp)) {  
    printf("Error again to write to a file\n");  
}  
return 0;  
}
```

```
C:\Users\Dell>a  
Provide feedback on the article: this article is good  
9 Bad code: Bad file descriptor  
  
Error writing to a file  
After clearing fp
```

Think about this - > If we comment clearerr(fp), what happens?

```
C:\Users\Dell>a  
Provide feedback on the article: good  
9 Bad code: Bad file descriptor  
  
Error writing to a file  
After clearing fp  
Error again to write to a file
```

The best practice is to use clearerr() **only after you have properly handled the error or end-of-file (EOF) condition**. This ensures that you're not ignoring unresolved issues that may affect the correctness of your program. This allows the file stream to be safely reused for further read or write operations without being blocked by previously set indicators.

Best practices to be followed during File handling

- Always Check File Pointers
- Validate Return Values of I/O Functions
- Use ferror() and feof() appropriately
- Clear Errors if needed
- Always Close Files
- Provide User-Friendly Error Messages
- Avoid Hard coding File Paths

Happy Error-Free File Handling implémentations!!